

DESIGN AND ANALYSIS OF ALGORITHMS

DAA LABORATORY – DAY 2 PROGRAMS

Algorithm Problems and Complexity Analysis

Name	Noelshaun Deril W
Register Number	192324174
Programme / Department	AI&DS
Academic Year	2026–2027

PROBLEM 1

FIRST PALINDROMIC STRING

Python Program

```
def first_palindrome(words):
    for word in words:
        if word == word[::-1]:
            return word
    return ""

words = ["abc", "car", "ada", "racecar", "cool"]
print("First palindromic string:", first_palindrome(words))
```

Execution Screenshot

EXECUTION SCREENSHOT

Paste execution screenshot here

PROBLEM 2

COUNT COMMON ELEMENTS IN TWO ARRAYS

Python Program

```
def count_common(nums1, nums2):  
    set1 = set(nums1)  
    set2 = set(nums2)  
    answer1 = sum(1 for x in nums1 if x in set2)  
    answer2 = sum(1 for x in nums2 if x in set1)  
    return [answer1, answer2]  
  
nums1 = [2, 3, 2]  
nums2 = [1, 2]  
print("Output:", count_common(nums1, nums2))
```

Execution Screenshot

EXECUTION SCREENSHOT

Paste execution screenshot here

PROBLEM 3

SUM OF SQUARES OF DISTINCT COUNTS

Python Program

```
def distinct_square_sum(nums):  
    total = 0  
    for i in range(len(nums)):  
        distinct = set()  
        for j in range(i, len(nums)):  
            distinct.add(nums[j])  
            count = len(distinct)  
            total += count * count  
    return total  
  
nums = [1, 2, 1]  
print("Output:", distinct_square_sum(nums))
```

Execution Screenshot

EXECUTION SCREENSHOT

Paste execution screenshot here

PROBLEM 4

COUNT EQUAL-VALUE PAIRS DIVISIBLE BY K

Python Program

```
def count_pairs(nums, k):  
    count = 0  
    for i in range(len(nums)):  
        for j in range(i + 1, len(nums)):  
            if nums[i] == nums[j] and (i * j) % k == 0:  
                count += 1  
    return count  
  
nums = [3, 1, 2, 2, 2, 1, 3]  
k = 2  
print("Number of valid pairs:", count_pairs(nums, k))
```

Execution Screenshot

EXECUTION SCREENSHOT

Paste execution screenshot here

PROBLEM 5

MAXIMUM ELEMENT IN AN ARRAY

Python Program

```
def find_maximum(arr):
    maximum = arr[0]
    for num in arr[1:]:
        if num > maximum:
            maximum = num
    return maximum

test_cases = [[1, 2, 3, 4, 5],
               [7, 7, 7, 7, 7],
               [-10, 2, 3, -4, 5]]

for arr in test_cases:
    print("Input:", arr)
    print("Maximum:", find_maximum(arr))
```

Execution Screenshot

EXECUTION SCREENSHOT

Paste execution screenshot here

PROBLEM 6

SORT THE LIST AND FIND MAXIMUM

Python Program

```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i]); i += 1
        else:
            result.append(right[j]); j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result

def maximum_after_sorting(arr):
    if not arr:
        return None
    return merge_sort(arr)[-1]

for arr in [[], [5], [3, 3, 3, 3, 3]]:
    result = maximum_after_sorting(arr)
    print("Input:", arr)
    print("Output:", "List is empty" if result is None else result)
```

Execution Screenshot

EXECUTION SCREENSHOT

Paste execution screenshot here

PROBLEM 7

UNIQUE ELEMENTS AND SPACE COMPLEXITY

Python Program

```
def unique_elements(arr):
    seen = set()
    result = []
    for num in arr:
        if num not in seen:
            seen.add(num)
            result.append(num)
    return result

tests = [[3, 7, 3, 5, 2, 5, 9, 2],
        [-1, 2, -1, 3, 2, -2],
        [1000000, 999999, 1000000]]

for arr in tests:
    print("Input:", arr)
    print("Unique elements:", unique_elements(arr))

# Space Complexity: O(n)
```

Execution Screenshot

EXECUTION SCREENSHOT

Paste execution screenshot here

PROBLEM 8

BUBBLE SORT

Python Program

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n - 1):
        swapped = False
        for j in range(n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True
        if not swapped:
            break
    return arr

arr = [5, 1, 4, 2, 8]
print("Sorted array:", bubble_sort(arr))

# Time Complexity: O(n) best case, O(n^2) average/worst case
```

Execution Screenshot

EXECUTION SCREENSHOT

Paste execution screenshot here

PROBLEM 9

BINARY SEARCH

Python Program

```
def binary_search(arr, key):
    low, high = 0, len(arr) - 1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == key:
            return mid
        elif arr[mid] < key:
            low = mid + 1
        else:
            high = mid - 1
    return -1

arr = [3, 4, 6, -9, 10, 8, 9, 30]
arr.sort()

for key in [10, 100]:
    pos = binary_search(arr, key)
    if pos != -1:
        print("Element", key, "is found at position", pos + 1)
    else:
        print("Element", key, "is not found")

# Time Complexity: O(log n)
```

Execution Screenshot

EXECUTION SCREENSHOT

Paste execution screenshot here

PROBLEM 10

MERGE SORT IN $O(n \log n)$

Python Program

```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i]); i += 1
        else:
            result.append(right[j]); j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result

nums = [5, 2, 8, 1, 3, 7]
print("Sorted array:", merge_sort(nums))

# Time Complexity:  $O(n \log n)$ 
# Auxiliary Space:  $O(n)$ 
```

Execution Screenshot

EXECUTION SCREENSHOT

Paste execution screenshot here